

User Agent Inspector — Technical Field Reference

Document Version: 1.0

Date: May 2026

Project: useragent-web

Repository: github.com/ammaleasy/useragent-web

Table of Contents

1. [Introduction](#)
 2. [Section 1: UA-String Values](#)
 - 2.1 Browser
 - 2.2 Rendering Engine
 - 2.3 Operating System
 - 2.4 Device Type
 - 2.5 CPU Architecture
 3. [Section 2: Browser Environment Information](#)
 - 3.1 Language
 - 3.2 Platform
 - 3.3 Cookies
 - 3.4 CPU Cores
 - 3.5 Screen Resolution
 - 3.6 Viewport
 - 3.7 Touch Support
 - 3.8 Timezone
 - 3.9 Device Memory
 - 3.10 Color Depth
 - 3.11 Color Scheme Preference
 - 3.12 Reduced Motion Preference
 - 3.13 Network Information
 - 3.14 Online Status
 - 3.15 Do Not Track
 - 3.16 PDF Viewer
 - 3.17 JavaScript Status
 4. [Summary Table](#)
 5. [References](#)
-

1. Introduction

This document provides a comprehensive technical reference for all fields detected and displayed by the **User Agent Inspector** web application. Each field entry includes its definition, the JavaScript API or method used for extraction, example values, and relevant technical notes.

The fields are organized into two categories:

- **UA-String Values** — Information parsed directly from the `navigator.userAgent` string, which corresponds to the HTTP `User-Agent` request header.
 - **Browser Environment Information** — Data obtained from various Web APIs that expose device and browser capabilities beyond what the UA string contains.
-

2. Section 1: UA-String Values

These values are extracted by parsing the `navigator.userAgent` property using regular expressions. The User-Agent string is a structured text field that browsers transmit with every HTTP request, containing product tokens, version numbers, and platform identifiers.

2.1 Browser

Attribute	Detail
Definition	The name and major version of the web browser in use.
Example Values	Chrome 125.0, Firefox 126.0, Edge 125.0, Safari 17.5
Data Source	<code>navigator.userAgent</code> — regex pattern matching on product tokens

Extraction Method:

```
const ua = navigator.userAgent;

// Ordered from most specific to least specific to avoid false matches
// (e.g., Edge UA contains "Chrome/" token)
const browsers = [
  { name: "Edge",          rx: /Edg(?:e|A|iOS)?\[([0-9.]+)\]/ },
  { name: "Samsung Browser", rx: /SamsungBrowser\[([0-9.]+)\]/ },
  { name: "Opera",          rx: /OPR\[([0-9.]+)\]|Opera\[([0-9.]+)\]/ },
  { name: "Chrome",         rx: /Chrome\[([0-9.]+)\]/ },
  { name: "Firefox",        rx: /Firefox\[([0-9.]+)\]/ },
  { name: "Safari",         rx: /Version\[([0-9.]+)\].*Safari/ },
  { name: "IE",             rx: /MSIE ([0-9.]+)|Trident\[([0-9.]+)\]/ },
];

let browser = { name: "Unknown", version: "-" };
for (const b of browsers) {
  const m = ua.match(b.rx);
  if (m) {
    browser = { name: b.name, version: m[1] || m[2] };
    break;
  }
}
```

Technical Notes:

The ordering of regex patterns is critical. Chromium-based browsers (Edge, Opera, Samsung Browser) include the `Chrome/` token in their UA string for backward compatibility. Therefore, more specific patterns must be evaluated before the generic `Chrome/` pattern.

2.2 Rendering Engine

Attribute	Detail
-----------	--------

Definition	The layout and rendering engine responsible for interpreting HTML, CSS, and producing the visual output.
Possible Values	WebKit / Blink, Gecko, Trident, Presto
Data Source	navigator.userAgent — engine token identification

Extraction Method:

```
let engine = "Unknown";
if (/Gecko\[0-9\]/.test(ua) && /Firefox/.test(ua)) engine = "Gecko";
else if (/AppleWebKit\[0-9\]/.test(ua)) engine = "WebKit / Blink";
else if (/Trident\/.test(ua)) engine = "Trident";
else if (/Presto\/.test(ua)) engine = "Presto";

const engineVersion = (ua.match(/(?:AppleWebKit|Gecko|Trident)\/([0-9.]+)/) || [])[1] || "-";
```

Technical Notes:

Engine	Used By
WebKit / Blink	Chrome, Edge, Safari, Opera (post-v15)
Gecko	Firefox
Trident	Internet Explorer
Presto	Opera (pre-v15, legacy)

Blink is a fork of WebKit; both report `AppleWebKit/` in the UA string for compatibility reasons.

2.3 Operating System

Attribute	Detail
Definition	The operating system and version running on the client device.
Possible Values	Windows 11, Windows 10, macOS, iPadOS, iOS, Android, Linux, ChromeOS
Data Source	navigator.userAgent comment block + navigator.maxTouchPoints (for iPad detection)

Extraction Method:

```
const osList = [
  { name: "Windows 11", rx: /Windows NT 10\.\.Win64/ },
  { name: "Windows 10", rx: /Windows NT 10\.\./ },
  { name: "Windows 8.1", rx: /Windows NT 6\.\.3/ },
  { name: "Windows 8", rx: /Windows NT 6\.\.2/ },
  { name: "Windows 7", rx: /Windows NT 6\.\.1/ },
  { name: "macOS", rx: /Mac OS X ([0-9_]+)/ },
  { name: "iOS", rx: /iPhone OS ([0-9_]+)/ },
```

```

    { name: "iPadOS",      rx: /iPad.*OS ([0-9]+)/ },
    { name: "Android",     rx: /Android ([0-9]+)/ },
    { name: "Linux",       rx: /Linux/ },
    { name: "ChromeOS",    rx: /CrOS/ },
  ];

  let os = { name: "Unknown", version: "-" };
  for (const o of osList) {
    const m = ua.match(o.rx);
    if (m) {
      os = { name: o.name, version: m[1] ? m[1].replace(/_/g, ".") : "-" };
      break;
    }
  }

  // Special case: iPad with "Request Desktop Website" enabled
  if (os.name === "macOS" && navigator.maxTouchPoints > 1) {
    os.name = "iPadOS";
    os.version = "(Desktop mode - version unavailable)";
  }

```

Technical Notes:

Since iPadOS 13 (2019), Safari on iPad defaults to "Request Desktop Website," causing the UA string to report `Macintosh; Intel Mac OS X` instead of `iPad`. The `navigator.maxTouchPoints` property provides differentiation: genuine macOS devices report `0`, while iPad reports `5`.

Windows NT Version Mapping:

NT Version	Product Name
10.0	Windows 10 / 11
6.3	Windows 8.1
6.2	Windows 8
6.1	Windows 7

2.4 Device Type

Attribute	Detail
Definition	The form factor classification of the client device.
Possible Values	Desktop, Mobile Phone, Tablet (iPad), Smart TV, Bot / Crawler
Data Source	<code>navigator.userAgent</code> keywords + <code>navigator.maxTouchPoints</code>

Extraction Method:

```

const isFakeDesktopIPad = /Macintosh/.test(ua) && navigator.maxTouchPoints > 1;

let deviceType = "Desktop";

```

```
if (isFakeDesktopIPad)                deviceType = "Tablet (iPad)";
else if (/Mobile|Android.*Mobile/.test(ua)) deviceType = "Mobile Phone";
else if (/iPad|Android(?:!.*Mobile)|Tablet/.test(ua)) deviceType = "Tablet";
else if (/TV|SmartTV|HbbTV/.test(ua))   deviceType = "Smart TV";
else if (/bot|crawl|spider/i.test(ua))  deviceType = "Bot / Crawler";
```

Technical Notes:

Device classification logic (evaluated in priority order):

- 1. Check for iPad desktop mode (Macintosh + touch points > 1)
- 2. Mobile keyword or Android.*Mobile → mobile phone
- 3. iPad , Tablet , or Android without Mobile → tablet
- 4. TV , SmartTV , HbbTV → smart television
- 5. bot , crawl , spider (case-insensitive) → automated crawler
- 6. None of the above → desktop computer

2.5 CPU Architecture

Attribute	Detail
Definition	The instruction set architecture of the device's CPU as reported in the UA string.
Possible Values	x86-64 (64-bit), x86 (32-bit), ARM, Unknown
Data Source	navigator.userAgent — platform tokens in the comment block

Extraction Method:

```
let arch = "Unknown";
if (/Win64|x64|x86_64|WOW64/.test(ua)) arch = "x86-64 (64-bit)";
else if (/Win32|i686|i386/.test(ua))   arch = "x86 (32-bit)";
else if (/arm|aarch64/i.test(ua))     arch = "ARM";
```

Technical Notes:

- WOW64 indicates a 32-bit process running on a 64-bit Windows system (Windows 32-bit on Windows 64-bit).
- Modern mobile devices (iOS, newer Android) use ARM architecture but may not always expose this in the UA string.
- aarch64 denotes 64-bit ARM (e.g., Apple Silicon Macs, newer Android devices).

3. Section 2: Browser Environment Information

These fields are obtained from JavaScript Web APIs that provide information about the device, display, network, and user preferences. They are independent of the User-Agent string.

3.1 Language

Attribute	Detail
-----------	--------

Definition	The user's preferred language(s) as configured in the browser settings. Corresponds to the HTTP Accept-Language header.
Example Values	en-US, th, ja
API	navigator.language, navigator.languages

Extraction Method:

```
const primaryLanguage = navigator.language;           // e.g., "en-US"
const allLanguages = navigator.languages;             // e.g., ["en-US", "en", "th"]
```

Technical Notes:

`navigator.language` returns the primary language preference. `navigator.languages` returns an ordered array reflecting the full language priority list, which matches the `Accept-Language` header sent with HTTP requests.

3.2 Platform

Attribute	Detail
Definition	A string identifying the operating system and CPU platform on which the browser is executing.
Example Values	MacIntel, Win32, Linux x86_64, iPhone
API	navigator.platform

Extraction Method:

```
const platform = navigator.platform;
```

Technical Notes:

This property is **deprecated** per the HTML Living Standard but remains widely supported. It may return misleading values (e.g., iPad in desktop mode reports `MacIntel`). The successor is the User-Agent Client Hints API (`navigator.userAgentData`), which is not yet universally supported.

3.3 Cookies

Attribute	Detail
Definition	Indicates whether the browser is configured to accept HTTP cookies.
Possible Values	Enabled (true), Disabled (false)
API	navigator.cookieEnabled

Extraction Method:

```
const cookiesEnabled = navigator.cookieEnabled;    // boolean
```

Technical Notes:

Returns `true` if the browser will accept cookies for the current document context. Note that this does not guarantee third-party cookies are enabled — browsers increasingly block those by default while keeping first-party cookies active.

3.4 CPU Cores

Attribute	Detail
Definition	The number of logical CPU processors available to the browser's execution context.
Example Values	4, 8, 10, 16
API	<code>navigator.hardwareConcurrency</code>

Extraction Method:

```
const logicalCores = navigator.hardwareConcurrency; // number
```

Technical Notes:

This value represents logical processors (including hyper-threading/SMT threads), not physical cores. It is useful for determining how many Web Workers to spawn for parallel computation. Some browsers may cap this value for privacy reasons.

3.5 Screen Resolution

Attribute	Detail
Definition	The total display dimensions in CSS pixels and the device pixel ratio (DPR).
Example Values	1920 × 1080, 2x DPR
API	<code>screen.width</code> , <code>screen.height</code> , <code>window.devicePixelRatio</code>

Extraction Method:

```
const screenWidth = screen.width;           // CSS pixels
const screenHeight = screen.height;         // CSS pixels
const dpr = window.devicePixelRatio;       // e.g., 2 for Retina displays
```

Technical Notes:

- CSS pixels ≠ physical pixels. A `devicePixelRatio` of 2 means each CSS pixel maps to a 2×2 grid of physical pixels.
- Retina/HiDPI displays typically report DPR of 2 or 3.
- The actual physical resolution is `screen.width × DPR` by `screen.height × DPR`.

3.6 Viewport

Attribute	Detail
Definition	The visible content area of the browser window, excluding toolbars, scrollbars, and browser chrome.
Example Values	1440 × 820 px
API	window.innerWidth, window.innerHeight

Extraction Method:

```
const viewportWidth = window.innerWidth;    // CSS pixels
const viewportHeight = window.innerHeight;  // CSS pixels
```

Technical Notes:

Distinct from `screen.width/height` which reports the full display. The viewport represents the actual rendering area available to the web page. This value changes when the user resizes the browser window or rotates a mobile device.

3.7 Touch Support

Attribute	Detail
Definition	The number of simultaneous touch contact points the device's input system supports.
Example Values	Yes (5 points), No
API	navigator.maxTouchPoints

Extraction Method:

```
const maxTouchPoints = navigator.maxTouchPoints;    // number (0 = no touch)
const hasTouch = maxTouchPoints > 0;
```

Technical Notes:

- Desktop computers without touch screens report `0`.
- iPads consistently report `5`.
- This property is instrumental in detecting iPad devices disguised as Macintosh (see Section 2.3).
- Some Windows laptops with touch screens will report non-zero values despite being classified as desktops.

3.8 Timezone

Attribute	Detail
-----------	--------

Definition	The IANA timezone identifier and UTC offset based on the system's regional settings.
Example Values	Asia/Bangkok, UTC+7:00
API	<code>Intl.DateTimeFormat().resolvedOptions().timeZone</code> , <code>Date.prototype.getTimezoneOffset()</code>

Extraction Method:

```
const timezone = Intl.DateTimeFormat().resolvedOptions().timeZone; //
"Asia/Bangkok"

const offsetMinutes = -new Date().getTimezoneOffset();           // minutes from
UTC
const offsetString = (offsetMinutes >= 0 ? "UTC+" : "UTC")
  + Math.floor(offsetMinutes / 60) + ":"
  + String(Math.abs(offsetMinutes % 60)).padStart(2, "0");       // "UTC+7:00"
```

Technical Notes:

`getTimezoneOffset()` returns the difference in **minutes** between UTC and local time (negative for east of UTC). The sign is inverted from intuitive expectation, hence the negation. The IANA identifier provides a human-readable name that accounts for daylight saving time rules.

3.9 Device Memory

Attribute	Detail
Definition	An approximate amount of device RAM, rounded to the nearest power of 2 for privacy protection.
Example Values	4 GB, 8 GB
API	<code>navigator.deviceMemory</code>
Browser Support	Chromium-based browsers only

Extraction Method:

```
const memoryGB = navigator.deviceMemory;           // 0.25, 0.5, 1, 2, 4, or 8
```

Technical Notes:

This API intentionally reduces precision to mitigate fingerprinting. Values are bucketed to powers of 2, capped at 8 GB maximum. A device with 6 GB RAM may report 8. Firefox and Safari do not implement this API; the property will be `undefined` in those browsers.

3.10 Color Depth

Attribute	Detail
Definition	The number of bits used to represent the color of a single pixel on the display.
Example Values	24-bit (True Color), 30-bit (HDR/Deep Color)
API	screen.colorDepth, screen.pixelDepth

Extraction Method:

```
const colorDepth = screen.colorDepth;           // bits per pixel (typically 24
or 30)
const pixelDepth = screen.pixelDepth;           // usually identical to
colorDepth
```

Technical Notes:

- 24-bit = 8 bits per channel × 3 channels (RGB) = 16,777,216 colors (True Color)
- 30-bit = 10 bits per channel × 3 channels = 1,073,741,824 colors (HDR displays)
- `pixelDepth` and `colorDepth` are functionally equivalent in modern browsers.

3.11 Color Scheme Preference

Attribute	Detail
Definition	The user's system-level preference for light or dark color scheme, as configured in OS settings.
Possible Values	Dark, Light
API	window.matchMedia("(prefers-color-scheme: dark)")

Extraction Method:

```
const prefersDarkMode = window.matchMedia("(prefers-color-scheme: dark)").matches;
// true = dark mode, false = light mode
```

Technical Notes:

This reflects the OS-level appearance setting (e.g., macOS Appearance, Windows Personalization > Colors, Android Dark theme). Web applications can use this to automatically match the user's preferred theme without requiring manual toggling.

3.12 Reduced Motion Preference

Attribute	Detail
Definition	Indicates whether the user has requested minimized motion and animation through OS accessibility settings.
Possible Values	Requested (reduce), No preference

API	<code>window.matchMedia("(prefers-reduced-motion: reduce)")</code>
-----	--

Extraction Method:

```
const prefersReducedMotion = window.matchMedia("(prefers-reduced-motion: reduce)").matches;
```

Technical Notes:

This accessibility setting is intended for users with vestibular motion disorders, epilepsy, or motion sensitivity. When enabled, web applications should:

- Disable or reduce CSS animations and transitions
- Avoid parallax scrolling effects
- Minimize auto-playing animated content

3.13 Network Information

Attribute	Detail
Definition	The estimated effective connection type and downlink bandwidth based on observed network performance.
Example Values	4G, 10 Mbps
API	<code>navigator.connection</code> (Network Information API)
Browser Support	Chromium-based browsers only

Extraction Method:

```
const connection = navigator.connection
  || navigator.mozConnection
  || navigator.webkitConnection;

if (connection) {
  const effectiveType = connection.effectiveType; // "slow-2g", "2g", "3g", "4g"
  const downlink = connection.downlink;          // Estimated bandwidth in Mbps
  const rtt = connection.rtt;                     // Round-trip time in
  milliseconds
  const saveData = connection.saveData;           // boolean: data saver mode
}
```

Technical Notes:

`effectiveType` is not the actual connection technology (WiFi vs. cellular) but rather a classification based on measured throughput and latency:

Effective Type	Downlink	RTT
slow-2g	< 50 Kbps	> 2000 ms

2g	< 70 Kbps	> 1400 ms
3g	< 700 Kbps	> 270 ms
4g	≥ 700 Kbps	≤ 270 ms

A slow WiFi connection may be classified as `3g` if its performance matches that profile.

3.14 Online Status

Attribute	Detail
Definition	Indicates whether the browser currently has network connectivity.
Possible Values	Online (<code>true</code>), Offline (<code>false</code>)
API	<code>navigator.onLine</code>

Extraction Method:

```
const isOnline = navigator.onLine; // boolean

// Event listeners for connectivity changes
window.addEventListener("online", () => { /* network restored */ });
window.addEventListener("offline", () => { /* network lost */ });
```

Technical Notes:

A return value of `true` does not guarantee internet access — it only confirms that a network interface is active. The device may be connected to a local network without internet routing. For reliable connectivity verification, an application-level health check (e.g., fetching a known endpoint) is recommended.

3.15 Do Not Track

Attribute	Detail
Definition	A user-configured signal requesting that websites refrain from cross-site tracking of the user's browsing activity.
Possible Values	Enabled (<code>"1"</code>), Disabled (<code>"0"</code>), Not set (<code>null/undefined</code>)
API	<code>navigator.doNotTrack</code>

Extraction Method:

```
const dnt = navigator.doNotTrack || window.doNotTrack;
// "1" = user requests no tracking
// "0" = user consents to tracking
// null/undefined = no preference expressed
```

Technical Notes:

DNT is a **voluntary signal** with no legal enforcement mechanism in most jurisdictions (notable exception:

certain EU interpretations under GDPR). Safari has removed DNT support entirely, citing its misuse as a fingerprinting vector. The W3C Tracking Protection Working Group was disbanded in 2019 without producing a final recommendation. The Global Privacy Control (GPC) header is emerging as a successor with stronger legal backing.

3.16 PDF Viewer

Attribute	Detail
Definition	Indicates whether the browser has a built-in capability to render PDF documents inline without external plugins.
Possible Values	Supported (true), Not supported (false)
API	<code>navigator.pdfViewerEnabled</code>

Extraction Method:

```
if (typeof navigator.pdfViewerEnabled !== "undefined") {  
  const hasPdfViewer = navigator.pdfViewerEnabled; // boolean  
}
```

Technical Notes:

This property was introduced as a replacement for the deprecated `navigator.plugins` and `navigator.mimeTypes` arrays. All major modern browsers (Chrome, Firefox, Edge, Safari) include built-in PDF rendering. The API returns `undefined` in older browsers that do not implement it.

3.17 JavaScript Status

Attribute	Detail
Definition	Confirms that JavaScript execution is enabled and functional in the browser.
Possible Values	Enabled (invariably <code>true</code> when detectable)
API	Implicit — determined by successful code execution

Extraction Method:

```
// No explicit detection required.  
// If this code executes, JavaScript is enabled.  
const jsEnabled = true;
```

Technical Notes:

JavaScript detection is inherently self-referential: if the detection code runs, JS is enabled. For graceful degradation when JS is disabled, the `<noscript>` HTML element should be used to provide fallback content.

4. Summary Table

#	Field	Source API / Method	Category
1	Browser	navigator.userAgent (regex)	UA-String
2	Rendering Engine	navigator.userAgent (regex)	UA-String
3	Operating System	navigator.userAgent (regex) + maxTouchPoints	UA-String
4	Device Type	navigator.userAgent (regex) + maxTouchPoints	UA-String
5	CPU Architecture	navigator.userAgent (regex)	UA-String
6	Language	navigator.language / navigator.languages	Browser Env
7	Platform	navigator.platform	Browser Env
8	Cookies	navigator.cookieEnabled	Browser Env
9	CPU Cores	navigator.hardwareConcurrency	Browser Env
10	Screen Resolution	screen.width / screen.height / devicePixelRatio	Browser Env
11	Viewport	window.innerWidth / window.innerHeight	Browser Env
12	Touch Support	navigator.maxTouchPoints	Browser Env
13	Timezone	Intl.DateTimeFormat().resolvedOptions().timeZone	Browser Env
14	Device Memory	navigator.deviceMemory	Browser Env
15	Color Depth	screen.colorDepth / screen.pixelDepth	Browser Env
16	Color Scheme	matchMedia("(prefers-color-scheme)")	Browser Env
17	Reduced Motion	matchMedia("(prefers-reduced-motion)")	Browser Env
18	Network Info	navigator.connection	Browser Env
19	Online Status	navigator.onLine	Browser Env
20	Do Not Track	navigator.doNotTrack	Browser Env
21	PDF Viewer	navigator.pdfViewerEnabled	Browser Env
22	JavaScript	Implicit (code execution)	Browser Env

5. References

1. MDN Web Docs — Navigator API: <https://developer.mozilla.org/en-US/docs/Web/API/Navigator>
2. MDN Web Docs — User-Agent string: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/User-Agent>
3. W3C — Network Information API: <https://wicg.github.io/netinfo/>
4. W3C — Device Memory API: <https://www.w3.org/TR/device-memory/>
5. HTML Living Standard — Navigator interface: <https://html.spec.whatwg.org/multipage/system-state.html>

6. Apple Developer — iPadOS Desktop-class browsing:

<https://developer.apple.com/documentation/safari-release-notes>

Prepared by the useragent-web Project — May 2026